

Autonomous Emergency Braking System (AEBS)

System Specification and Formal Abstraction

Heba Al Kayed

Abstract

This document formally specifies the dynamics of the AEBS case study. It is divided into three primary sections. **Part I** defines the concrete simulation environment, modelled as a Discrete-Time Stochastic Hybrid System (dt-SHS), detailing the physical plant, sensor noise model, and controller logic. **Part II** details the formal abstraction pipeline, which discretises the continuous state space into an Interval Markov Decision Process (IMDP) for formal verification using the PRISM model checker [1]. The abstraction is grounded in the *Approximate Abstractions of Stochastic Hybrid Systems* procedure [2], extended with theoretical coverage of probabilistic reachability from both [3] and the *Adaptive and Sequential Gridding Procedures* paper [4]. A physically motivated refinement to the epsilon error bound exploits the separable kernel structure of the AEBS plant to obtain informative transition intervals. **Part III** presents the formal verification results and analyses the theoretical guarantees in relation to the foundational literature. All implementation listings are collected in the Appendix.

Contents

1	System Architecture	3
2	Vehicle Plant Dynamics	3
2.1	State Space Definition	3
2.2	Discrete-Time Update Law	3
3	Perception and Sensor Noise Model	4
3.1	Observation Function	4
3.2	Measurement Uncertainty and Truncation	5
3.3	Perception Modelling Limitations	5
4	Controller Logic	5
5	Formal Abstraction Methodology	6
5.1	Correspondence to the Approximate Abstractions Paper	6
6	Abstraction Algorithms	7
7	The Epsilon Error Bound	8
7.1	What Epsilon Is and Why It Is Needed	8
7.2	The Naïve Global Formula and Why It Fails	9
7.3	Per-Dimension Epsilon: The Corrected Formula	9
7.4	Structural Limitation: Vacuous Lower Bounds	9
8	Grid Configuration	10
9	Theoretical Deviations and Soundness	10
9.1	Physically Motivated Refinements (Soundness Preserved)	11
9.2	Structural Differences (Scope of Guarantees Adjusted)	11
10	Verification Setup	11
11	Verification Results	12
11.1	Interpretation: Nominal Cruising	12
11.2	Structural Integrity Check	12
12	Comparison with Theoretical Framework	12
12.1	Approximate Abstractions of Stochastic Hybrid Systems [2]	12
12.2	Adaptive and Sequential Gridding Procedures [4]	12
12.3	Deviation Summary	13
13	Computational Cost and Scalability	13
A	Implementation Listings	13

Part I: Concrete System Dynamics

1 System Architecture

The system is modelled as a closed-loop simulation consisting of four distinct components:

- **Vehicle Plant ($\mathcal{P}$):** A discrete-time physical model of the ego vehicle.
- **Traffic Environment ($\mathcal{E}$):** Manages the dynamics of the lead vehicle and scenario configuration.
- **Perception System ($\mathcal{S}$):** Acts as a stochastic observer function $\mathbf{y} = h(\mathbf{s})$.
- **Controller ($\mathcal{C}$):** A deterministic reactive policy mapping observations to control inputs.

2 Vehicle Plant Dynamics

The vehicle dynamics are implemented in the `VehiclePlant` class. The system state evolves in discrete time steps of $\Delta t = 0.1$ s.

2.1 State Space Definition

The state vector is defined as $\mathbf{s}_k = [x_k, v_k, a_k]^\top$, where:

- x_k — Position (or Gap) at step k .
- v_k — Velocity (or Closing Speed) at step k .
- a_k — Longitudinal Acceleration at step k .

The system supports two coordinate references. In the **World Frame**, x denotes the absolute position on the track. In the **Relative Frame**, x denotes the gap to the obstacle (decreasing as the vehicle approaches). The relative frame is used exclusively for formal abstraction because it places the crash condition at $x = 0$, making it a natural absorbing boundary.

2.2 Discrete-Time Update Law

The physics engine implements a **sequential forward-chaining update**: each state variable is computed using the most recently updated value from the same time step. This improves physical consistency by ensuring velocity uses the latest acceleration, and position uses the latest velocity.

Step 1 — Acceleration Update (Actuator Lag)

$$a_{k+1} = a_k + \alpha \cdot (u_k - a_k) \tag{1}$$

where:

- a_{k+1} — Actual longitudinal acceleration of the ego vehicle at step $k + 1$.
- a_k — Actual longitudinal acceleration at the previous step k .
- u_k — Target acceleration requested by the controller at step k , in m/s^2 .
- α — Actuator lag coefficient ($\alpha = 0.5$), modelling hydraulic and mechanical delay as a discrete-time first-order low-pass filter.

Real vehicles cannot change acceleration instantaneously. A command for emergency braking at $u_k = -9.8 \text{ m/s}^2$ from rest ($a_k = 0$) produces $a_{k+1} = -4.9 \text{ m/s}^2$ in the first step, asymptotically approaching the target over subsequent steps. This mirrors the physical accumulation of hydraulic brake pressure.

Step 2 — Velocity Update

$$v_{k+1} = \max(0, v_k + a_{k+1} \cdot \Delta t) \quad (2)$$

where:

- v_{k+1} — Closing speed at step $k + 1$.
- v_k — Closing speed at step k .
- a_{k+1} — Newly computed acceleration from Equation (1).
- Δt — Time step duration (0.1 s).

The $\max(0, \cdot)$ clamp is a physical boundary constraint: a vehicle that decelerates to a complete stop does not begin moving backwards, guaranteeing $v_{k+1} \geq 0$ at all times.

Step 3 — Position Update

$$x_{k+1} = \begin{cases} x_k + v_{k+1} \cdot \Delta t & \text{if world frame} \\ x_k - v_{k+1} \cdot \Delta t & \text{if relative frame} \end{cases} \quad (3)$$

where:

- x_{k+1} — Gap to the lead vehicle at step $k + 1$ (relative frame).
- x_k — Gap at step k .
- v_{k+1} — Newly computed velocity from Equation (2).

In the relative frame, a positive closing speed v_{k+1} reduces the gap over Δt , hence the subtraction. The Python implementation of Equations (1)–(3) is provided in Listing 1 of the Appendix.

3 Perception and Sensor Noise Model

The `PerceptionSystem` implements the observation function $h : \mathcal{S} \rightarrow \mathcal{Y} \cup \{\emptyset\}$, mapping the ground truth state $\mathbf{s}_k$ to a noisy measurement $\mathbf{y}_k = [\tilde{x}, \tilde{v}]^\top$ or a null detection $\emptyset$. The model captures two distinct uncertainty types that mirror real-world automotive sensor systems.

3.1 Observation Function

$$\mathbf{y}_k = \begin{cases} \emptyset & \text{if } x_k > d_{\max} \quad (\text{out of range}) \\ \emptyset & \text{with probability } P_{FN} \quad (\text{missed detection}) \\ [\tilde{x}, \tilde{v}]^\top & \text{otherwise} \end{cases} \quad (4)$$

where:

- $\emptyset$ — Null detection: the sensor suite failed to perceive the target.
- x_k — Ground truth relative gap to the target.
- $d_{\max}$ — Maximum physical detection range (100 m), a conservative bound for the operational design domain.
- P_{FN} — Probability of a false negative (missed detection), modelling a DNN classifier failure modelled as a Bernoulli process.
- $\tilde{x}, \tilde{v}$ — Noisy observed relative gap and closing velocity.

3.2 Measurement Uncertainty and Truncation

For successful detections, the system introduces aleatoric uncertainty via additive zero-mean Gaussian noise:

$$\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, \Sigma), \quad \Sigma = \begin{bmatrix} \sigma_{pos}^2 & 0 \\ 0 & \sigma_{vel}^2 \end{bmatrix} \quad (5)$$

where:

- $\mathbf{w}_k$ — Sampled noise vector at step k .
- Σ — Diagonal covariance matrix, assuming gap and velocity noise are statistically independent.
- σ_{pos} — Standard deviation of gap measurement noise (0.5 m).
- σ_{vel} — Standard deviation of velocity measurement noise (0.1 m/s).

The baseline values represent the lower end of typical measurement tolerances for commercial 77 GHz FMCW automotive radars [5, 6]. The observed quantities are:

$$\tilde{x} = \max(0, x_k + w_{pos}) \quad (6)$$

$$\tilde{v} = v_k + w_{vel} \quad (7)$$

where w_{pos} and w_{vel} are the scalar components of $\mathbf{w}_k$. The $\max(0, \cdot)$ clamp in Equation (6) prevents physically impossible negative distances, causing $\tilde{x}$ to follow a rectified Gaussian rather than a standard Normal distribution. This discrepancy is most significant when x_k approaches zero. The Python implementation is provided in Listing 2 of the Appendix.

3.3 Perception Modelling Limitations

- **No false positives:** the model can miss objects but never hallucinates obstacles.
- **Independent noise:** gap and velocity errors are assumed uncorrelated, which may underestimate co-degradation of sensor returns in adverse conditions.
- **No systematic bias:** noise is strictly zero-mean; sensor drift and misalignment are not modelled.
- **Stateless execution:** each sensor read is independent. There is no Kalman filter or temporal tracking logic.

4 Controller Logic

The controller implements a reactive decision policy. It does not access the ground truth state $\mathbf{s}_k$; it operates entirely on the noisy observation $\mathbf{y}_k$ and an internal dead-reckoning velocity estimate. The decision logic branches on observed closing velocity: a low-speed approach ($v_{rel} < 5.0$ m/s) is governed by spatial distance thresholds, while a high-speed approach is governed by TTC thresholds. This dual-trigger design reflects the standard multi-stage collision mitigation algorithms used in current automotive research and national safety standards [7]. The Python implementation is provided in Listing 3 of the Appendix.

Table 1: Controller benchmark parameters

Parameter	Symbol	Industry Mode	Safe Mode
<i>Time Thresholds</i>			
Warning TTC	TTC_{warn}	2.6 s	6.0 s
Braking TTC	TTC_{brake}	1.6 s	5.0 s
Emergency TTC	TTC_{crit}	1.0 s	4.0 s
<i>Distance Thresholds</i>			
Warning Dist	d_{warn}	10.0 m	15.0 m
Braking Dist	d_{brake}	6.0 m	10.0 m
Emergency Dist	d_{crit}	2.0 m	5.0 m
<i>Actuation</i>			
Brake Force	u_{brake}	-4.0 m/s^2	-4.0 m/s^2
Emergency Force	u_{crit}	-9.8 m/s^2	-8.0 m/s^2

Note: The Warning thresholds (TTC_{warn} , d_{warn}) are used strictly for state labelling and formal verification safety monitoring, not to trigger physical braking.

Part II: Abstracted System Dynamics

5 Formal Abstraction Methodology

The core challenge is that the AEBS state space is continuous and infinite. The pipeline resolves this by replacing the continuous system with a finite model that preserves probabilistic safety guarantees. The abstraction is grounded in the *Approximate Abstractions of Stochastic Hybrid Systems* paper [2] for the construction procedure and the *Adaptive and Sequential Gridding Procedures* paper [4] for the probabilistic invariance framework and tighter error bounds. The probabilistic reachability semantics of the IMDP queries are established in [1, 3].

At a high level, the pipeline works as follows. First, the continuous state space is covered with a finite grid of rectangular cells. For each cell, the probability of transitioning to each neighbouring cell is computed by integrating the stochastic plant dynamics over the cell boundaries — this is the *nominal* transition probability. Because the cell is represented by a single point (its centroid), this estimate is only exact at the centroid; from any other point inside the cell it may differ. The *abstraction error bound* ϵ quantifies this worst-case discrepancy. Each nominal probability is inflated into an interval $[p - \epsilon, p + \epsilon]$, and the resulting structure is an Interval Markov Decision Process (IMDP) — formally an encoding of the Markov Set-Chain constructed in [2].

PRISM resolves the interval non-determinism adversarially [8]: $P_{\max}$ queries use the worst-case transition assignment at every step, yielding a certified upper bound on crash probability. $P_{\min}$ queries use the best-case assignment. Both are sound over-approximations of the true continuous-system probabilities.

5.1 Correspondence to the Approximate Abstractions Paper

The implementation maps directly to the construction in Section IV of [2], with the absorbing boundary states following the explicit ϕ -state design in Algorithm 1 of [4]. Algorithm 1 below formalises the full pipeline; Algorithms 2 and 3 detail the two computationally intensive sub-steps.

6 Abstraction Algorithms

Algorithm 1: Finite Modular IMDP Abstraction of a dt-SHS

Inputs: Continuous dt-SHS with state space X , action space U , stochastic kernel T_x ; per-dimension noise σ ; Lipschitz constants L_t, L_q .

1. **Partition.** Cover X with n non-overlapping rectangular cells $\{X_i\}$. Select the geometric centroid $\bar{x}_i$ of each cell as its representative point. Augment the state set with two absorbing boundary states: x_{crash} ($gap \leq 0$) and x_{safe_sink} (out-of-domain mass).
2. **Compute transitions.** For each cell X_i and each action $\bar{u}$:
 - (a) Apply the deterministic plant dynamics to the centroid: $\bar{x}' = f(\bar{x}_i, \bar{u})$ — the *nominal successor*.
 - (b) Integrate the separable stochastic kernel over each neighbouring cell to obtain nominal transition probabilities $\hat{p}_j$ (Algorithm 2).
 - (c) Compute the error bound ϵ from the Lipschitz constants and the cell half-width in the stochastic dimension (Section 7).
 - (d) Form the IMDP interval: $[\max(0, \hat{p}_j - \epsilon), \min(1, \hat{p}_j + \epsilon)]$.
 - (e) Route remaining Gaussian tail mass (outside the grid) to x_{safe_sink} with the same interval width.
3. **Label.** Assign atomic propositions to each cell based on kinematic thresholds: **crash**, **braking**, **emergency**, or **safe**.
4. **Compose.** Assemble the labelled plant IMDP with the controller and perception modules using PRISM synchronisation labels **time_step**, **perceive**, and **control** (Listing 6).

Output: A PRISM-compatible modular IMDP $\hat{\Sigma}$.

Algorithm 2: Separable Kernel Integration for One Cell–Action Pair

Inputs: Nominal successor $\bar{x}'$, per-dimension noise $\sigma = [0, 0, \sigma_{v_{lead}}]$, grid $\mathcal{G}$, error bound ϵ .

1. **Identify stochastic dimensions.** For the AEBS plant, only v_{lead} (dimension 2) is stochastic ($\sigma_d > 0$). The gap and v_{ego} dimensions are deterministic ($\sigma_d = 0$).
2. **Define search region.**
 - Stochastic dimensions: search within 4σ of $\bar{x}'$ (captures $> 99.99\%$ of Gaussian mass).
 - Deterministic dimensions: restrict to the single cell containing $\bar{x}'_d$ (an indicator — probability 1 in the correct cell, 0 elsewhere).
3. **Compute joint probabilities.** For each candidate target cell X_j in the search region, the transition probability is a product of per-dimension terms:

$$\hat{p}(X_j \mid \bar{x}', \sigma) = \prod_{d=1}^3 K_d(X_j^d \mid \bar{x}'_d, \sigma_d) \quad (8)$$

where each per-dimension factor K_d is:

$$K_d = \begin{cases} \Phi\left(\frac{high_d - \bar{x}'_d}{\sigma_d}\right) - \Phi\left(\frac{low_d - \bar{x}'_d}{\sigma_d}\right) & \sigma_d > 0 \quad (\text{Gaussian CDF integral}) \\ \mathbf{1}[low_d \leq \bar{x}'_d < high_d] & \sigma_d = 0 \quad (\text{indicator function}) \end{cases} \quad (9)$$

4. **No pruning.** Retain all candidates within the search region. The *Approximate Abstractions* paper [2] requires full kernel integration: discarding tail probability mass would violate the normalisation that the ϵ bound relies upon.
5. **Route boundary mass.** Assign crash boundary mass (gap ≤ 0) to x_{crash} and remaining unaccounted mass to x_{safe_sink} , each wrapped in $[\max(10^{-6}, p - \epsilon), \min(1, p + \epsilon)]$.

Symbol key for Equations (8)–(9):

- Φ — Standard Normal CDF.
- $high_d, low_d$ — Upper and lower boundary of cell X_j in dimension d .
- $\bar{x}'_d$ — Component of the nominal successor in dimension d .
- σ_d — Per-dimension noise standard deviation (0 for deterministic dimensions).
- $\mathbf{1}[\cdot]$ — Indicator function: 1 if the condition holds, 0 otherwise.

Output: Full set of IMDP transition intervals from cell X_i under action $\bar{u}$.
The Python implementation is in Listing 5 of the Appendix.

7 The Epsilon Error Bound

7.1 What Epsilon Is and Why It Is Needed

Replacing the continuous state space with a finite grid introduces an approximation error: the nominal transition probability is computed at the cell centroid, but the true system may be anywhere inside the cell. The bound ϵ guarantees that the true probability cannot differ from the centroid estimate by more than ϵ , regardless of where in the cell the system actually is.

The *Approximate Abstractions* paper [2] derives ϵ using the Lipschitz continuity of the stochastic kernel: if transition probabilities cannot change faster than L per unit of distance in state space, and the furthest any point in the cell can be from the centroid is $\delta/2$, then the worst-case error is bounded by $L \cdot \delta/2$.

7.2 The Naïve Global Formula and Why It Fails

The straightforward application of the *Approximate Abstractions* bound uses the full Euclidean cell diameter across all three dimensions:

$$\delta_{\text{global}} = 2 \|(0.5 \text{ m}, 0.25 \text{ m/s}, 0.125 \text{ m/s})\|_2 \approx 1.12 \quad (10)$$

where the Euclidean norm incoherently mixes metres (gap) and m/s (velocities). Combined with:

$$L_q = \frac{2}{\sigma_{v_{lead}} \sqrt{2\pi}} \approx 4.0 \text{ [units: (m/s)}^{-1}] \quad (11)$$

$$\epsilon_{\text{naive}} = (L_t + L_q) \cdot \delta_{\text{global}} \approx 5.7 \gg 1 \quad (12)$$

where:

- L_q — Lipschitz constant of the Gaussian kernel: maximum rate of change of the transition probability per unit shift in the kernel mean, equal to $2/(\sigma\sqrt{2\pi})$, the PDF peak.
- L_t — Lipschitz constant of the deterministic dynamics ($L_t \approx 1.05$, the spectral norm of the dynamics Jacobian $\partial f/\partial x$).
- $\sigma_{v_{lead}}$ — Standard deviation of the stochastic kernel on v_{lead} (0.2 m/s at baseline noise).

With $\epsilon > 1$, every transition interval collapses to $[10^{-6}, 1.0]$: formally sound but informationally void.

7.3 Per-Dimension Epsilon: The Corrected Formula

The AEBS plant has a **separable kernel**: noise enters only through v_{lead} (dimension 2). The gap and v_{ego} dimensions are deterministic — their per-dimension kernel is an indicator function, which has zero Lipschitz constant with respect to perturbations within the same cell. Therefore, only the stochastic dimension’s half-width enters the error bound:

$$\delta_{\text{stochastic}} = \frac{\text{resolution}[v_{lead}]}{2} = \frac{0.25 \text{ m/s}}{2} = 0.125 \text{ m/s} \quad (13)$$

$$\boxed{\epsilon = (L_t + L_q) \cdot \delta_{\text{stochastic}}} \quad (14)$$

where $\delta_{\text{stochastic}}$ is the half-width of the grid cell in the noisy dimension (v_{lead} only). This is the per-dimension form of the bound from [2], valid for separable kernels where noise enters a single dimension. The Python implementation is in Listing 4 of the Appendix.

Verified at runtime:

```
sigma_per_dim=[0.    0.    0.20006], delta_stochastic=0.1250, L_t=1.052, L_q=3.988,
epsilon=0.6300
```

With $\epsilon = 0.63 < 1$, transition intervals are non-trivial: upper bounds vary meaningfully across cells and the P_{max} query yields genuine safety information.

7.4 Structural Limitation: Vacuous Lower Bounds

Despite $\epsilon < 1$, all interval lower bounds remain zero. For the central cell (where the Gaussian is centred), with $\sigma_{v_{lead}} = 0.2 \text{ m/s}$ and cell width $w = 0.25 \text{ m/s}$:

$$p_{\text{central}} = \Phi\left(\frac{w/2}{\sigma_{v_{lead}}}\right) - \Phi\left(\frac{-w/2}{\sigma_{v_{lead}}}\right) = \Phi\left(\frac{0.125}{0.2}\right) - \Phi\left(-\frac{0.125}{0.2}\right) \approx 0.47 \quad (15)$$

$$\max(0, p_{\text{central}} - \epsilon) = \max(0, 0.47 - 0.63) = 0 \quad (16)$$

where:

- p_{central} — Probability mass of the Gaussian kernel within the central target cell.

- $w/2 = \delta_{\text{stochastic}} = 0.125 \text{ m/s}$ — Half cell width in the stochastic dimension.
- Φ — Standard Normal CDF.

This is provably unavoidable at any grid resolution under the Lipschitz gradient bound: as cell width w decreases, $p_{\text{central}} \sim w/(\sigma\sqrt{2\pi})$ while $\epsilon \sim (L_t + 2/(\sigma\sqrt{2\pi})) \cdot w/2$; their ratio is always less than 1 for any realistic L_t and σ . Consequently, P_{min} queries return 0 trivially and carry no formal content.

The *Adaptive and Sequential Gridding Procedures* paper [4] addresses this with a range-based bound (their equation 3.11) that measures the actual variation of the kernel over the source cell directly rather than bounding it via a global gradient:

$$\epsilon_{ij} = \max_{\bar{s} \in A_j} \left[\max_{s \in A_i} t_s(\bar{s}|s) - \min_{s \in A_i} t_s(\bar{s}|s) \right] \quad (17)$$

where:

- A_i, A_j — Source and target partition cells.
- $t_s(\bar{s}|s)$ — Stochastic kernel density at target point $\bar{s}$ given source point s .

This bound is computed cell-pair by cell-pair and is expected to produce non-trivial lower bounds, resolving the vacuous P_{min} issue. Implementation is planned for the next development branch.

8 Grid Configuration

The `medium` preset is used for all verification runs. The v_{lead} resolution of 0.25 m/s was chosen specifically to keep $\delta_{\text{stochastic}} = 0.125 \text{ m/s}$ and thus $\epsilon < 1$.

Table 2: Grid parameters (`medium` preset)

Parameter	Value
Gap resolution	1.0 m
v_{ego} resolution	0.5 m/s
v_{lead} resolution	0.25 m/s
Gap bounds	[0, 100] m
Velocity bounds	[0, 30] m/s
Acceleration bounds	[-15, 10] m/s ²
Total states	$\approx 622,261 + 1 \text{ sink}$
PRISM file size	$\approx 640 \text{ MB}$

The acceleration bounds were widened to [-15, 10] m/s² to prevent probability tunnelling: without adequate boundary margin, Gaussian tail mass would leak outside the grid and be silently lost rather than routed to $x_{\text{safe_sink}}$. Available presets are listed in Table 3.

Table 3: Available grid presets

Preset	Resolution (m, m/s, m/s)	States (approx)	Purpose
<code>debug</code>	(5.0, 2.0, 1.0)	~ 500	Sanity checks
<code>coarse</code>	(2.0, 1.0, 0.5)	$\sim 15\text{K}$	Fast iteration
<code>medium</code>	(1.0, 0.5, 0.25)	$\sim 622\text{K}$	Current baseline
<code>fine</code>	(0.5, 0.1, 0.1)	$\sim 50\text{M}+$	Intractable locally

9 Theoretical Deviations and Soundness

The pipeline implements the core construction of the *Approximate Abstractions* paper [2]. Deviations fall into two categories.

9.1 Physically Motivated Refinements (Soundness Preserved)

Per-dimension epsilon. The *Approximate Abstractions* paper [2] uses a global Euclidean cell diameter. This implementation uses the separability argument of Section 7: since noise enters only the v_{lead} dimension, only $\delta_{v_{lead}}$ contributes to the Lipschitz error. The resulting ϵ is strictly tighter than the global form and remains a valid upper bound under the separable kernel assumption.

Absorbing boundary states. The *Approximate Abstractions* paper [2] assumes a compact recurrent domain from which the system never escapes. The AEBS plant can crash ($\text{gap} \leq 0$) or safely separate ($\text{gap} \gg 0$), requiring absorbing states. This is an intended design element: the *Adaptive and Sequential Gridding Procedures* paper [4] explicitly constructs an absorbing state ϕ in Algorithm 1 to handle probability mass leaving the bounded safe region.

9.2 Structural Differences (Scope of Guarantees Adjusted)

Non-ergodicity. The *Approximate Abstractions* paper [2] requires the system to be ψ -irreducible and strongly aperiodic (Assumption 2) for the asymptotic convergence theorems (Theorems 1–3). The AEBS model violates this because crash and safe-sink states are absorbing. This does *not* invalidate $P_{\max}$ reachability results: reachability verification requires only single-step soundness of the IMDP intervals, not long-run ergodic behaviour. The ergodicity condition is specific to steady-state distribution tracking, which is not the query being computed.

Vacuous lower bounds. As shown in Section 7, all lower probability bounds are zero. This is a structural conservatism of the Lipschitz gradient bound applied to Gaussian kernels, not a correctness issue. $P_{\max}$ results remain valid. The range-based bound from [4] (Equation (17)) is the planned fix.

Part III: Formal Verification Results

10 Verification Setup

The generated IMDP was verified using the PRISM model checker’s explicit engine [1] on the DICE compute cluster (betel.inf.ed.ac.uk), with a 64 GB Java heap and PRISM 4.8.1. The model declares the initial state as an undetermined integer constant `start_s`, so any grid state can be queried without regenerating the model. All five scenarios are submitted in a single invocation (Listing 7) so that the 8-hour model build is paid only once.

Table 4: Scenario state IDs (`medium` preset)

Scenario	Physics (<code>gap</code> , <code>v</code> , <code>a</code>)	State ID
Nominal Cruising	(40.0 m, 15.0 m/s, 0.0)	249530
Warning Threshold	(25.0 m, 15.0 m/s, 0.0)	157115
Critical Braking	(15.0 m, 15.0 m/s, 0.0)	95505
Imminent Collision	(10.0 m, 20.0 m/s, 0.0)	65710
Post-Collision	(0.0 m, 0.0 m/s, 0.0)	0

11 Verification Results

Table 5: Formal verification: $P_{\max}(\text{crash})$ per scenario ($\epsilon = 0.63$, `medium` preset, $\sigma_{v_{lead}} = 0.2 \text{ m/s}$)

Scenario	Kinematics	$P_{\max}$	Build time	Reachable states
Nominal Cruising	40 m, 15 m/s	0.0	8.4 h	17,434
Warning Threshold	25 m, 15 m/s	<i>pending</i>	—	—
Critical Braking	15 m, 15 m/s	<i>pending</i>	—	—
Imminent Collision	10 m, 20 m/s	<i>pending</i>	—	—
Post-Collision	0 m, 0 m/s	<i>pending</i>	—	—

The remaining four queries are currently running on the DICE cluster as a single invocation and will be populated upon completion.

11.1 Interpretation: Nominal Cruising

$P_{\max}(\text{crash}) = 0.0$ from state 249530 means that even the worst-case adversary — assigning transition probabilities to their upper-bound values within the IMDP intervals at every step — cannot reach a crash state from the nominal cruising condition (40 m gap, 15 m/s). This is a formal one-sided safety certificate: the true crash probability is provably ≤ 0 under this abstraction.

The 17,434 reachable states (out of 622K total) confirm the nominal trajectory is confined to a compact region of state space, consistent with a well-controlled highway cruise. The uniformly deterministic structure (max choices = avg choices = 1.00) confirms the controller selects a single action throughout the reachable subgraph: the controller mapping from this initial state is fully deterministic.

Note that $P_{\min}$ queries trivially return 0 due to the structural limitation of Section 7 and carry no formal content.

11.2 Structural Integrity Check

Model checking `Pmax=?[F "safe_sink"]` from the crash state (state 0) returned exactly 0.0, confirming that the crash state is strictly absorbing: probability mass cannot leak from the crash state into the safe sink post-collision.

12 Comparison with Theoretical Framework

12.1 Approximate Abstractions of Stochastic Hybrid Systems [2]

This paper provides the construction procedure used here. Its main convergence theorems bound the error between the IMDP’s probability distribution and that of the true continuous system over time, requiring ergodicity (Assumption 2 of [2]). Since the AEBS system is structurally non-ergodic due to its absorbing states, these theorems do not directly apply. However, as established in Section 9, this does not affect the validity of $P_{\max}$ reachability results.

12.2 Adaptive and Sequential Gridding Procedures [4]

This follow-up paper is the closer theoretical fit for three reasons.

1. **Absorbing states are by design.** Algorithm 1 of [4] explicitly includes an absorbing state ϕ for probability mass leaving the bounded safe set — the direct analogue of `safe_sink`. Absorbing states are therefore an intended design element of the framework.
2. **The error bound is over reachability probabilities.** Theorem 3.2 of [4] bounds $|p_{s_0}(A) - p_{p_0}(A_p)| \leq \gamma\delta$, directly quantifying the gap between the true and abstract reachability probabilities — precisely the relevant guarantee for `Pmax=?[F "crash"]`.
3. **Tighter bounds resolve the lower-bound issue.** The range-based bound of Equation 3.11 in [4] (Equation (17) here) would produce non-trivial lower bounds and is the planned next implementation step.

12.3 Deviation Summary

Table 6: Deviations from the *Approximate Abstractions* paper [2] and their consequences

Aspect	Approximate stractions [2]	Ab-	This Implementation	Consequence
Epsilon formula	Global Euclidean $\delta^n \cdot L$		$(L_t + L_q) \cdot \delta_{\text{stoch}}$	Tighter; separability argument justifies validity
Absorbing states	None (compact recurrent domain)		<code>crash</code> , <code>safe_sink</code>	Consistent with Adaptive Gridding [4]
Ergodicity	Required for Theorems 1–3		Not satisfied	Steady-state bounds inapplicable; reachability unaffected
Out-of-domain mass	Cannot occur		Routed to <code>safe_sink</code>	Consistent with ϕ state in [4]
Lower bounds	Can be positive		Always 0	$P_{\min}$ trivially 0; range-based fix planned [4]
Verification query	Steady-state distribution		<code>Pmax=?[F "crash"]</code>	Ergodicity not required for reachability

13 Computational Cost and Scalability

Table 7: Verification computational cost

Machine	Heap	Model build	Model check	States
Local PC (PRISM 4.9)	12 GB	~16 min	<1 s	~300K (old grid)
DICE betel (PRISM 4.8.1)	64 GB	8.4 h	<1 s	622K / 17K reachable

Model construction entirely dominates the cost. Once built, value iteration to convergence takes under one second for any query. The 622K-state model on a 64 GB machine already requires 8+ hours, illustrating the state explosion that motivates approximate abstraction methods. Scaling to the `fine` preset (~50M states) is currently intractable.

A Implementation Listings

All code is from the committed `epsilon-fix` branch. Listings are referenced in the main text at the points where they are first discussed.

```

1 # Step 1      Acceleration (first-order actuator lag)
2 self.actual_acceleration = (self.actual_acceleration
3   + self.alpha * (action_acceleration - self.actual_acceleration))
4
5 # Step 2      Velocity using latest acceleration
6 self.actual_velocity += self.actual_acceleration * self.dt
7 if self.actual_velocity < 0.0:      # physical clamp: no reverse motion
8     self.actual_velocity = 0.0
9     self.actual_acceleration = 0.0
10
11 # Step 3      Gap using latest velocity (relative frame: positive v closes gap)
12 self.x -= self.actual_velocity * self.dt

```

Listing 1: Sequential plant dynamics update (`vehicle_plant.py`), implements Equations (1)–(3)

```

1 if real_gap > 100.0:      # deterministic out-of-range
2     return False, 100.0, 0.0
3 if np.random.rand() < self.fn_rate:      # Bernoulli false-negative
4     return False, real_gap, real_v_rel
5

```

```

6 gap_noise = np.random.normal(0, self.pos_noise) # sigma_pos = 0.5 m
7 vel_noise = np.random.normal(0, self.vel_noise) # sigma_vel = 0.1 m/s
8
9 obs_gap = max(0.0, real_gap + gap_noise) # clamped >= 0
10 obs_v_rel = real_v_rel + vel_noise # unclamped

```

Listing 2: Sensor noise injection (perception.py), implements Equations (4)–(7)

```

1 # Dead-reckoning velocity estimate
2 self.est_velocity = max(0.0, self.est_velocity + self.prev_accel_req * self.dt)
3
4 # Time-to-collision (guard against division by zero)
5 ttc = obs_gap / obs_v_rel if obs_v_rel > 0.0 else float('inf')
6
7 # Dual-trigger decision (distance at low speed, TTC at high speed)
8 if obs_v_rel < 5.0:
9     if obs_gap < self.dist_emergency: return self.acc_emergency, 'emergency_brake'
10    elif obs_gap < self.dist_brake: return self.acc_brake, 'brake'
11 else:
12     if ttc < self.ttc_emergency: return self.acc_emergency, 'emergency_brake'
13    elif ttc < self.ttc_brake: return self.acc_brake, 'brake'
14 return 0.0, 'coast'

```

Listing 3: Reactive controller decision tree (controller.py)

```

1 # sigma_per_dim = [0.0, 0.0, sigma_v_lead]
2 stochastic_dims = np.where(sigma_per_dim > 0)[0]
3
4 if len(stochastic_dims) > 0:
5     # Use the stochastic dim with the largest cell half-width (conservative)
6     stochastic_dim = stochastic_dims[np.argmax(grid_resolution[stochastic_dims])]
7     delta_stochastic = grid_resolution[stochastic_dim] / 2.0
8 else:
9     delta_stochastic = np.min(grid_resolution) / 2.0 # fully deterministic fallback
10
11 epsilon = (L_t + L_q) * delta_stochastic
12 # Verified: epsilon = 0.6300 (medium preset, sigma = 0.2 m/s)

```

Listing 4: Per-dimension epsilon computation (discretizer.py), implements Equation (14)

```

1 for idx in itertools.product(*ranges):
2     prob = 1.0
3     for d in range(3):
4         low, high = bins[d][idx[d]], bins[d][idx[d]+1]
5         mean = next_center[d]
6
7         if sigma_per_dim[d] > 0:
8             # Gaussian CDF integral stochastic dimension (v_lead)
9             prob *= fast_norm_cdf(high, mean, sigma_per_dim[d]) \
10                    - fast_norm_cdf(low, mean, sigma_per_dim[d])
11         else:
12             # Indicator kernel deterministic dimensions (gap, v_ego)
13             prob *= 1.0 if low <= mean < high else 0.0
14
15 # NO PRUNING: full 4-sigma radius retained (required by Abate et al. 2011)

```

Listing 5: Separable Gaussian kernel integration (discretizer.py), implements Equations (8)–(9)

```

1 def _write_turn_module(self, f):
2     f.write("module Turn\n")
3     f.write("    t : [1..3] init 1;\n")
4     f.write("    [time_step] (t=1) -> (t'=2);\n") # Plant executes
5     f.write("    [perceive] (t=2) -> (t'=3);\n") # Perception reads state
6     f.write("    [control] (t=3) -> (t'=1);\n") # Controller selects action
7     f.write("endmodule\n\n")

```

Listing 6: PRISM synchronisation turn module (prism-generator.py)

```

1 nice -n 15 ~/prism-4.8.1-linux64-x86/bin/prism ~/modular_system.prism \
2 -javamaxmem 200g \
3 -pctl 'Pmax=? [ F "crash" ]' \

```

```
4 -const start_s=249530,157115,95505,65710,60 \
5 > ~/pmax_all.txt 2>&1
```

Listing 7: PRISM verification command: all scenarios in a single model build

References

- [1] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008.
- [2] Alessandro Abate, Alessandro D’Innocenzo, and Maria D. Di Benedetto. Approximate abstractions of stochastic hybrid systems. *IEEE Transactions on Automatic Control*, 56(11):2688–2693, 2011.
- [3] Alessandro Abate, Maria Prandini, John Lygeros, and Shankar Sastry. Probabilistic reachability and safety for controlled discrete time stochastic hybrid systems. *Automatica*, 44(11):2724–2734, 2008.
- [4] Sadeh Esmaeil Zadeh Soudjani and Alessandro Abate. Adaptive and sequential gridding procedures for the abstraction and verification of stochastic processes. *SIAM Journal on Applied Dynamical Systems*, 12(2):921–956, 2013.
- [5] Jürgen Hasch, Eray Topak, Reinhard Schnabel, Thomas Zwick, Robert Weigel, and Christian Waldschmidt. Millimeter-wave technology for automotive radar sensors in the 77 GHz frequency band. *IEEE Transactions on Microwave Theory and Techniques*, 60(3):845–860, 2012.
- [6] Texas Instruments. Introduction to mmwave sensing: FMCW radars. Technical Report SWRA553A, Texas Instruments, 2020.
- [7] Fang Lai et al. A comprehensive review of autonomous emergency braking systems. *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [8] Morteza Lahijanian, Sean B. Andersson, and Calin Belta. Formal verification and synthesis for discrete-time stochastic systems. *IEEE Transactions on Automatic Control*, 60(8):2031–2045, 2015.